GUIDE-1：读懂 sudoku.c

这份指南写给几乎没有学过操作系统的读者，目标是让你能逐行看懂 Lab02 Part I 的 `sudoku.c`：一个用多线程检查数独答案是否正确的程序。实验要求见 `Lab02.md`。

阅读前提：会一点 C 语言（变量、`if`/`for`、函数、数组）。指针、`malloc`、函数指针等较难的写法，会在用到的地方单独解释。

阅读建议：把 `sudoku.c` 打开放在旁边对照着看。第 1、2 节打基础，第 3 节看全貌，第 4 节逐段读代码，第 5 节动手运行，第 6 节解答常见疑问。

1. 程序要解决什么问题

数独是一个 9×9 的方格，可以看成 9 行、9 列，以及 9 个 3×3 的小方块（下文称为**子格**）。一个正确的数独答案必须同时满足：

- 每一行都恰好包含 1～9 各一次；
- 每一列都恰好包含 1～9 各一次；
- 每个 3×3 子格都恰好包含 1～9 各一次。

一共 9 + 9 + 9 = 27 个区域，全部合格，答案才正确。

`sudoku.c` 要做的事情很简单：读入一个填好的数独（81 个整数），判断它是否正确，打印结论。

这个实验的重点不是检查算法，而是**用多线程完成检查**：把检查工作分给多个线程同时进行。本程序采用实验文档建议的方案，一共 11 个工作线程：

线程编号	负责检查
0	全部 9 行
1	全部 9 列
2 ～ 10	各负责 1 个 3×3 子格

2. 看懂代码需要的操作系统知识

这一节只讲读懂 `sudoku.c` 必需的概念。

2.1 程序、进程、线程

- **程序**：磁盘上的一个文件，例如编译出来的 `build/sudoku`。它只是一串指令，自己不会运行。
- **进程**：程序运行起来之后的实体。在终端输入 `./build/sudoku` 时，操作系统会创建一个进程：给它分配内存，把程序装进去，然后让 CPU 开始执行。
- **线程**：进程里真正在执行代码的"执行流"。每个进程启动时只有一个线程，叫**主线程**，它从 `main()` 开始执行。主线程可以再创建新的线程（本程序称为**工作线程**），每个新线程去执行一个指定的函数。

多个线程是**同时**运行的：在多核 CPU 上，它们可以在不同的核上真正并行；即使只有一个核，操作系统也会让它们快速轮流执行，看起来就像同时在跑。**哪个线程先跑、每次跑多久，由操作系统决定，程序员无法预知。**

2.2 线程之间共享什么

同一个进程里的所有线程：

- **共享**：全局变量（定义在所有函数外面的变量）、用 `malloc` 申请的内存、程序代码。一个线程修改了全局变量，其他线程能看到。
- **各自私有**：函数里的局部变量。每个线程都有自己的一块内存区域（叫**栈**）来存放局部变量。

所以，线程 2～10 这 9 个线程虽然同时执行同一个函数 `check_subgrid`，它们各自的局部变量（`data`、`index`）也互不干扰。

本程序正是利用"全局变量是共享的"这一点：把数独放进一个全局数组，让所有线程都能读；再用另一个全局数组收集各线程的检查结果。

2.3 Linux 上的线程工具：pthread

Linux 上创建和管理线程的标准接口叫 **POSIX 线程库**，简称 **pthread**（POSIX 是类 Unix 操作系统共同遵守的一套接口标准）。使用时，代码里要写 `#include <pthread.h>`；编译时传统上还要加 `-pthread` 选项（本项目的编译命令见 5.1 节，在较新的 glibc 上不加也能链接）。

本程序只用到其中一个类型和两个函数。

`pthread_t`：**线程的编号牌**。 创建线程时，系统把新线程的编号写进一个 `pthread_t` 变量；以后想等这个线程结束，就凭这个编号。

`pthread_create`：**创建一个新线程**。

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

参数	含义	本程序传入的值
thread	新线程的编号写到哪里	&workers[i]
attr	线程属性，NULL 表示使用默认设置	NULL
start_routine	新线程要执行的函数	check_rows、check_columns 或 check_subgrid
arg	传给这个函数的参数，**只能是一个指针**	一个 parameters 结构体的地址

成功时返回 0，失败时返回非 0 的错误码。

调用成功后，新线程就会开始执行 `start_routine(arg)`，而调用者**不会等它**，会立即继续往下执行。从这一刻起，两个线程同时在跑。

`pthread_join`：**等待一个线程结束**。

```
int pthread_join(pthread_t thread, void **retval);
```

调用者会停在这一行（称为**阻塞**），直到编号为 `thread` 的线程执行完毕才继续往下走。`retval` 用来接收线程函数的返回值，本程序用不到，传 `NULL`。

线程函数的固定格式。交给 `pthread_create` 执行的函数必须是这个样子：

```
void* 函数名(void* 参数名)
```

也就是：接收一个 `void *` 参数，返回一个 `void *`。

> **C 语法：`void *`**
>
> `void *` 是"可以指向任意类型数据的指针"。pthread 库不知道你想给线程传什么数据，所以统一用 `void *`。线程函数拿到之后，要先把它转换回真实的类型（本程序中是 `parameters *`），才能读取里面的内容。

2.4 数据竞争，以及本程序为什么不需要锁

如果两个线程同时访问同一个变量，并且其中至少一个在**写**，结果就可能出错，因为它们的执行顺序不可预知。这叫**数据竞争**（data race）。常见的解决办法是**加锁**（互斥锁，mutex），保证同一时刻只有一个线程能访问这个变量。

本程序在设计上避开了数据竞争，所以**不需要任何锁**：

1. **数独数组** `sudoku`：主线程在创建工作线程**之前**就把它填好了，之后所有线程都只读不写。多个线程同时读同一份数据是安全的。
2. **结果数组** `result`：线程 `i` 只写 `result[i]`，不会有两个线程写同一格。
3. 主线程等所有工作线程都**结束以后**，才去读 `result`。

POSIX 标准还保证：调用 `pthread_create` 之前写好的数据，新线程一定能看到；线程结束前写入的数据，`pthread_join` 返回后调用者一定能看到。所以上面这个顺序是可靠的。

3. 程序全貌

3.1 一个比喻

程序里的东西	比喻
进程	一间教室
主线程（执行 `main`）	组长
11 个工作线程	11 个组员
全局数组 `sudoku`	写在黑板上的数独，所有人都能看
`parameters` 结构体	组长发给组员的任务纸条，写着"从第几行、第几列开始检查"
全局数组 `result`	墙上的登记表，共 11 格，每个组员只在自己那格填 1（合格）或 0（不合格）
`pthread_create`	组长叫一个组员开工
`pthread_join`	组长等某个组员干完活
`free(data)`	组员看完纸条后把它扔掉

3.2 代码由哪些部分组成

按在文件中出现的顺序：

代码	作用
`#include`、`#define`	引入库函数；定义常量 `SIZE`（9）和 `NUM_THREADS`（11）
`parameters` 结构体	传给线程的参数：起始行、起始列
`int sudoku[SIZE][SIZE]`	全局变量：数独本身
`int result[NUM_THREADS]`	全局变量：每个线程的检查结果
`region_is_valid()`	检查一个矩形区域是否恰好包含 1～9
`check_rows()`	线程 0 执行的函数：检查全部行
`check_columns()`	线程 1 执行的函数：检查全部列
`check_subgrid()`	线程 2～10 执行的函数：检查一个子格
`create_worker()`	准备参数并创建一个线程
`main()`	主线程：读入 → 创建线程 → 等待 → 汇总并打印

C 语言要求函数先定义后使用，所以 `main` 写在文件最后；但程序是**从 `main` 开始执行**的。第 4 节按执行顺序讲解。

3.3 运行流程

```
main thread                          worker threads
──────────                           ──────────────
1. read 81 numbers into sudoku

2. create worker 0   ──────────→     worker 0 : check all rows      → result[0]
   create worker 1   ──────────→     worker 1 : check all columns   → result[1]
   create worker 2   ──────────→     worker 2 : check one subgrid   → result[2]
   ...                               ...
   create worker 10  ──────────→     worker 10: check one subgrid   → result[10]

3. join worker 0 .. 10               (the 11 workers run at the same time,
   (wait until all have finished)     in an order decided by the OS)

4. read result[0..10], print the verdict
```

左边是主线程的 4 步，右边是 11 个工作线程。图中 create 对应 `pthread_create`，join 对应 `pthread_join`。

注意第 2 步：主线程每创建一个线程，就马上去创建下一个，并不等它干完。所以很可能主线程还在创建线程 5，线程 0 就已经检查完了。

4. 按执行顺序读代码

4.1 开头：注释、头文件和常量

文件开头的英文注释大意是：数独答案校验器。从标准输入读入一个 9×9 的数独答案（81 个用空白分隔的整数），用 11 个工作线程校验：线程 0 检查每一行是否包含 1～9，线程 1 检查每一列，线程 2～10 各检查一个 3×3 子格（从左到右、从上到下）。最后是编译命令和运行方法。

接着是：

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 9
#define NUM_THREADS 11
```

头文件	本程序用到的函数
pthread.h	pthread_create、pthread_join
stdio.h	printf、scanf、fprintf、perror
stdlib.h	malloc、free、exit
string.h	strerror

`#define SIZE 9` 的意思是：编译之前，把代码里所有的 `SIZE` 替换成 `9`。给数字起个名字，代码更好读，也更好改。

4.2 共享数据：结构体和两个全局数组

```c
/* structure for passing data to threads */
typedef struct {
    int row;
    int column;
} parameters;

int sudoku[SIZE][SIZE];

/* result[i] is set by worker thread i: 1 = its region is valid, 0 = invalid */
int result[NUM_THREADS];
```

两条注释的意思分别是"用来给线程传递数据的结构体"和"`result[i]` 由第 i 个工作线程填写：1 表示它负责的区域合格，0 表示不合格"。

为什么需要 `parameters`？ `pthread_create` 只能给新线程传**一个**指针，但我们要告诉线程**两个**数：从哪一行、哪一列开始检查。办法是把这两个数装进一个结构体，再把结构体的地址传过去。

> **C 语法：`typedef struct { ... } parameters;`**
> 定义一个名叫 `parameters` 的新类型，它包含两个 `int` 成员 `row` 和 `column`。之后就可以像 `int` 一样用 `parameters` 来声明变量。

两个全局数组定义在所有函数外面，是全局变量，所有线程都能访问（见 2.2）：

- `sudoku`：存放数独。`sudoku[0][0]` 是左上角，`sudoku[8][8]` 是右下角。**代码里的行号、列号都从 0 开始**，只有打印给人看时才加 1，变成 1～9。
- `result`：登记表。全局数组在程序启动时会自动初始化为全 0。

下标	由谁写入	含义
`result[0]`	线程 0	9 行是否全部合格
`result[1]`	线程 1	9 列是否全部合格
`result[2]` ～ `result[10]`	线程 2 ～ 10	各自负责的子格是否合格

4.3 main 第 1 步：读入数独

`main` 先声明了几个局部变量：

```
pthread_t workers[NUM_THREADS];
int valid = 1;
int i, j;
```

`workers` 是 11 个线程编号牌，`valid` 用来汇总最终结论。然后读入数独：

```
for (i = 0; i < SIZE; i++) {
    for (j = 0; j < SIZE; j++) {
        if (scanf("%d", &sudoku[i][j]) != 1) {
            fprintf(stderr,
                    "Error: expected %d integers on standard input\n",
                    SIZE * SIZE);
            return EXIT_FAILURE;
        }
    }
}
```

- 两层循环按"一行一行、每行从左到右"的顺序读 81 个整数，存进 `sudoku[i][j]`。
- `scanf("%d", &x)` 从**标准输入**读一个整数存进 `x`，成功时返回 1。标准输入默认是键盘，也可以改成从文件读（见 5.3）。`&` 是取地址：`scanf` 需要知道把数存到哪里。
- 如果返回值不是 1，说明输入提前结束或遇到了非数字，就向 `stderr`（专门输出错误信息的通道）打印提示，然后 `main` 返回 `EXIT_FAILURE`（值为 1），表示程序异常结束。

这一步只有主线程在运行，工作线程还不存在，所以写 `sudoku` 不会和任何线程冲突。

4.4 main 第 2 步：创建 11 个线程

```
create_worker(&workers[0], check_rows, 0, 0);
create_worker(&workers[1], check_columns, 0, 0);
for (i = 0; i < SIZE; i++)
    create_worker(&workers[2 + i], check_subgrid, (i / 3) * 3, (i % 3) * 3);
```

`create_worker` 的四个参数依次是：编号牌放在哪里、新线程要执行的函数、起始行、起始列。

- 线程 0 执行 `check_rows`，起点 (0, 0)；
- 线程 1 执行 `check_columns`，起点 (0, 0)；
- 线程 2～10 **都执行** `check_subgrid`，但起点不同，分别是 9 个子格的左上角。

子格的起点是这样算出来的：C 语言里两个整数相除只保留整数部分（`5 / 3` 等于 1），`%` 是取余数（`5 % 3` 等于 2）。当循环变量 `i` 从 0 走到 8 时：

i	起始行 (i/3)*3	起始列 (i%3)*3	线程编号 2+i	对应子格（从 1 开始数）
0	0	0	2	行 1-3，列 1-3
1	0	3	3	行 1-3，列 4-6
2	0	6	4	行 1-3，列 7-9
3	3	0	5	行 4-6，列 1-3
4	3	3	6	行 4-6，列 4-6
5	3	6	7	行 4-6，列 7-9
6	6	0	8	行 7-9，列 1-3
7	6	3	9	行 7-9，列 4-6
8	6	6	10	行 7-9，列 7-9

画在数独上就是：

	列 1-3	列 4-6	列 7-9
行 1-3	线程 2	线程 3	线程 4
行 4-6	线程 5	线程 6	线程 7
行 7-9	线程 8	线程 9	线程 10

再看 `create_worker` 内部：

```
void create_worker(pthread_t* tid, void* (*routine)(void*), int row,
                   int column) {
    parameters* data = (parameters*)malloc(sizeof(parameters));
    int err;

    if (data == NULL) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }
    data->row = row;
    data->column = column;

    err = pthread_create(tid, NULL, routine, data);
    if (err != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(err));
        exit(EXIT_FAILURE);
    }
}
```

它做了三件事：

1. **申请一张纸条**：`malloc(sizeof(parameters))` 向系统申请一块刚好放得下一个 `parameters` 的内存，返回这块内存的地址。申请失败会返回 `NULL`，这时用 `perror` 打印失败原因，再用 `exit` 结束整个程序。
2. **在纸条上写起点**：`data->row = row;` 和 `data->column = column;`。
3. **创建线程**：`pthread_create(tid, NULL, routine, data)` 创建新线程，让它执行 `routine(data)`，并把新线程的编号写进 `*tid`（也就是 `main` 里的 `workers[i]`）。返回值不是 0 表示失败，这时用 `strerror` 把错误码翻译成文字打印出来，然后结束程序。

`create_worker` 一返回，主线程就去创建下一个线程，**不会等刚创建的线程干完活**。

> **C 语法：`malloc` 和 `free`**
>
> `malloc(n)` 申请 n 个字节的内存，返回它的地址；这块内存会一直有效，直到调用 `free` 把它还回去。`sizeof(parameters)` 是一个 `parameters` 占用的字节数。前面的 `(parameters*)` 把返回的 `void *` 转换成 `parameters*`，在 C 语言里这个转换可写可不写。

> **C 语法：`->`**
>
> `data` 是指向结构体的指针，`data->row` 表示"`data` 所指向的结构体的 `row` 成员"，等价于 `(*data).row`。

> **C 语法：函数指针 `void* (*routine)(void*)`**
>
> 读作："`routine` 是一个指针，指向一个参数为 `void *`、返回值为 `void *` 的函数"，正好是线程函数的固定格式。C 语言中，函数名后面不加括号就代表这个函数的地址，所以 `main` 里可以直接写 `create_worker(&workers[0], check_rows, 0, 0)`，意思是"让新线程去执行 `check_rows`"。

至于纸条为什么必须用 `malloc` 申请、而不能用局部变量，见第 6 节问题 1。

4.5 工作线程在做什么

线程 0 执行的 `check_rows`：

```
void* check_rows(void* param) {
    parameters* data = (parameters*)param;
    int valid = 1;
    int i;

    for (i = data->row; i < SIZE && valid; i++)
        valid = region_is_valid(i, 0, 1, SIZE);

    result[0] = valid;
    free(data);
    return NULL;
}
```

逐行看：

1. `void* check_rows(void* param)`：线程函数的固定格式。函数名前面的 `void*` 是返回值类型，并不表示 `check_rows` 本身是指针。`param` 就是 `pthread_create` 的第 4 个参数，也就是纸条的地址，只是类型变成了 `void*`。
2. `parameters* data = (parameters*)param;`：把 `void*` 转回 `parameters*`，之后才能用 `data->row` 读纸条。
3. `int valid = 1;`：先假设所有行都合格。
4. `for` 循环：从第 `data->row` 行（也就是第 0 行）开始，一行一行地调用 `region_is_valid` 检查（4.6 节详解）。循环条件 `i < SIZE && valid` 的意思是：一旦发现某一行不合格（`valid` 变成 0），就立刻停止，后面的行不必再查。
5. `result[0] = valid;`：把结论填进登记表里属于自己的那一格。
6. `free(data);`：纸条用完了，归还这块内存。
7. `return NULL;`：线程函数返回，这个线程就结束了。

线程 1 执行的 `check_columns` 结构完全一样，只是改成逐列检查，并把结论写进 `result[1]`：

```
    for (j = data->column; j < SIZE && valid; j++)
        valid = region_is_valid(0, j, SIZE, 1);

    result[1] = valid;
```

线程 2～10 执行的 `check_subgrid`：

```
void* check_subgrid(void* param) {
    parameters* data = (parameters*)param;
    int index = 2 + (data->row / 3) * 3 + data->column / 3;

    result[index] = region_is_valid(data->row, data->column, 3, 3);
    free(data);
    return NULL;
}
```

9 个线程同时执行这同一个函数，但每个线程拿到的纸条不同，局部变量也各有一份（见 2.2），所以互不干扰。

`index` 这一行根据子格的起点，算出"我是几号线程、该写 `result` 的哪一格"，也就是把 4.4 节那张表反过来查。例如起点为第 3 行、第 6 列的线程：`2 + (3/3)*3 + 6/3 = 2 + 3 + 2 = 7`，正好是表中负责"行 4-6，列 7-9"的线程 7。

4.6 核心检查函数 `region_is_valid`

```
/* Return 1 if the height x width block whose top-left cell is (row, column)
 * contains each of the digits 1-9 exactly once, otherwise 0. */
int region_is_valid(int row, int column, int height, int width) {
    int seen[SIZE + 1] = {0};
    int i, j;

    for (i = row; i < row + height; i++) {
        for (j = column; j < column + width; j++) {
            int value = sudoku[i][j];

            if (value < 1 || value > SIZE || seen[value]) return 0;
            seen[value] = 1;
        }
    }
    return 1;
}
```

注释的意思是：如果左上角在 (row, column)、高 height 行、宽 width 列的区域恰好包含 1～9 各一次，返回 1，否则返回 0。这是一个普通的 C 函数，和线程无关。

参数描述一个矩形。 行、列、子格都可以看成矩形，所以一个函数就能检查三种区域：

调用方式	检查的区域
`region_is_valid(i, 0, 1, SIZE)`	第 i 行：1 行高、9 列宽
`region_is_valid(0, j, SIZE, 1)`	第 j 列：9 行高、1 列宽
`region_is_valid(r, c, 3, 3)`	左上角在 (r, c) 的 3×3 子格

`seen` 数组是一张签到表： `seen[v]` 为 1 表示数字 `v` 已经出现过。数组长度为 10，直接用下标 1～9 对应数字 1～9（下标 0 不用）；`= {0}` 把它全部初始化为 0。

两层循环依次取出区域里的每个数 `value`：

- 如果 `value` 不在 1～9 之间，不合格，返回 0；
- 如果 `seen[value]` 已经是 1，说明这个数重复出现，不合格，返回 0；
- 否则签到：`seen[value] = 1`。

9 个数都通过，就返回 1。为什么这样就够了？区域里正好有 9 个数，每个都在 1～9 之间并且互不相同，那它们只能恰好是 1～9 各一个。

举例：第 1 行 `6 2 4 5 3 9 1 8 7` 能全部签到，返回 1。如果某一行是 `6 2 4 5 3 9 1 8 6`，读到最后那个 6 时 `seen[6]` 已经是 1，返回 0。

> **C 语法：`||` 的"短路"**
>
> `A || B || C` 从左往右计算，只要有一个为真，就不再计算后面的。所以当 `value` 超出范围时，程序不会去访问 `seen[value]`，避免了数组越界。这三个条件的顺序不能随便调换。

4.7 main 第 3 步：等待所有线程结束

```
    for (i = 0; i < NUM_THREADS; i++) pthread_join(workers[i], NULL);
```

主线程按 0 到 10 的顺序逐个等待。如果某个线程早已结束，对应的 `pthread_join` 会立刻返回。循环结束时，11 个线程一定都已结束，`result` 的 11 个格子也都已填好。

第二个参数传 `NULL`，表示不需要线程函数的返回值：各线程都是 `return NULL`，真正的结果已经写在 `result` 里了。

4.8 main 第 4 步：汇总结果并打印

```
    for (i = 0; i < NUM_THREADS; i++) {
        printf("Thread %2d: %-7s  ", i, result[i] ? "valid" : "invalid");
        if (i == 0) {
            printf("all rows\n");
        } else if (i == 1) {
            printf("all columns\n");
        } else {
            int row = ((i - 2) / 3) * 3;
            int column = ((i - 2) % 3) * 3;

            printf("subgrid rows %d-%d, columns %d-%d\n", row + 1, row + 3,
                   column + 1, column + 3);
        }
        if (!result[i]) valid = 0;
    }

    printf("The Sudoku solution is %s.\n", valid ? "valid" : "invalid");
    return 0;
```

- 对每个线程，打印它的编号、结论，以及它负责的区域。子格的起点用和 4.4 节相同的公式算出（这里的 `i - 2` 就是创建线程时的循环变量），再加 1 换成人习惯的 1～9。
- 只要有一格是 0，`valid` 就变成 0。最后，11 格全是 1 才判定数独正确，否则判定不正确。`main` 返回 0 表示程序正常结束。

> **C 语法：本段用到的写法**
>
> - `条件 ? A : B`：条件为真取 A，否则取 B。`result[i] ? "valid" : "invalid"` 就是"是 1 打印 valid，是 0 打印 invalid"。
> - `%2d`：整数至少占 2 个字符宽，不足时在左边补空格，所以数字 0 会打印成"空格加 0"，和两位数 10 对齐。
> - `%-7s`：字符串至少占 7 个字符宽，`-` 表示靠左对齐、右边补空格，所以 `valid` 后面会补 2 个空格，和 `invalid` 一样宽。
> - `!result[i]`：`result[i]` 为 0 时为真。

5. 编译、运行和看懂输出

5.1 编译

```
mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sudoku.c" -o "build/sudoku" --static -lm
```

这条命令先生成 `build` 目录，再编译。各部分的含义：

部分	含义
`mkdir -p build`	创建 build 目录；-p 表示目录已存在时不报错
`&&`	前一条命令成功后才执行后一条
`gcc`	C 语言编译器
`-std=c99`	按 C99 标准编译
`-O2`	开启二级优化，让程序运行得更快
`-Wall`	打开常用的警告，帮你发现可疑的代码
`-fno-asm`	禁用 asm 关键字（本程序没有使用内联汇编）
`"sudoku.c"`	源代码文件；引号可以省略，写成 sudoku.c 效果一样
`-o "build/sudoku"`	把可执行文件输出到 build/sudoku
`--static`	静态链接，生成的可执行文件不依赖系统的动态库
`-lm`	链接数学库（本程序没有直接用到，保留在命令中不影响结果）

没有任何输出就说明编译成功，`build` 目录下会多出一个 `sudoku` 文件，运行时写 `./build/sudoku`。

5.2 准备输入

新建一个文本文件 `puzzle.txt`，写入 81 个整数，数与数之间用空格或换行隔开都可以。下面是实验文档里的例子：

```
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
```

5.3 运行

```
./build/sudoku < puzzle.txt
```

`<` 叫**输入重定向**：把 `puzzle.txt` 的内容当作程序的标准输入，相当于替你在键盘上敲进去。也可以只运行 `./build/sudoku`，然后手动输入 81 个数（可以分多行），输满 81 个并按回车后，程序就会继续。

输出：

```
Thread  0: valid    all rows
Thread  1: valid    all columns
Thread  2: valid    subgrid rows 1-3, columns 1-3
Thread  3: valid    subgrid rows 1-3, columns 4-6
Thread  4: valid    subgrid rows 1-3, columns 7-9
Thread  5: valid    subgrid rows 4-6, columns 1-3
Thread  6: valid    subgrid rows 4-6, columns 4-6
Thread  7: valid    subgrid rows 4-6, columns 7-9
Thread  8: valid    subgrid rows 7-9, columns 1-3
Thread  9: valid    subgrid rows 7-9, columns 4-6
Thread 10: valid    subgrid rows 7-9, columns 7-9
The Sudoku solution is valid.
```

每一行对应 `result` 的一格：`Thread 0` 表示所有行都合格，`Thread 1` 表示所有列都合格，接下来 9 行分别对应 9 个子格。最后一行是总结论。

5.4 动手试一个错误的数独

把 `puzzle.txt` 第 5 行正中间的 `4`（即 `sudoku[4][4]`）改成 `5`，第 5 行变成：

```
9 5 8 2 5 7 3 6 1
```

再运行一次：

```
Thread  0: invalid  all rows
Thread  1: invalid  all columns
Thread  2: valid    subgrid rows 1-3, columns 1-3
Thread  3: valid    subgrid rows 1-3, columns 4-6
Thread  4: valid    subgrid rows 1-3, columns 7-9
Thread  5: valid    subgrid rows 4-6, columns 1-3
Thread  6: invalid  subgrid rows 4-6, columns 4-6
Thread  7: valid    subgrid rows 4-6, columns 7-9
Thread  8: valid    subgrid rows 7-9, columns 1-3
Thread  9: valid    subgrid rows 7-9, columns 4-6
Thread 10: valid    subgrid rows 7-9, columns 7-9
The Sudoku solution is invalid.
```

被改的格子同时属于第 5 行、第 5 列和正中间的子格，这三个区域里原本都已经有一个 5，现在 5 出现了两次，所以线程 0、1、6 报告不合格，其余区域不受影响。你可以自己改别的格子，先预测哪些线程会报错，再运行验证。

6. 常见疑问

问题 1：纸条为什么要用 `malloc` 申请？在 `create_worker` 里定义一个局部变量 `parameters data;`，再把 `&data` 传给线程不行吗？

不行。局部变量只在函数执行期间有效：`create_worker` 一返回，`data` 所在的内存就会被回收，下一次调用 `create_worker` 很可能复用同一块内存，写进另一个线程的起点。而新线程什么时候真正开始读纸条是不确定的，它可能读到已经失效或被改写的数据，结果检查了错误的区域。

用 `malloc` 申请的内存会一直有效，直到被 `free`。每个线程都拿到一张属于自己的纸条，读完后由线程自己 `free`，既安全又不浪费内存。

问题 2：如果删掉 `pthread_join` 那个循环，会怎样？

主线程会在工作线程还没干完时就去读 `result`，这时有些格子还是初始值 0，可能把正确的数独误判为错误。更糟的是，`main` 一返回整个进程就结束了，还没跑完的线程会被直接终止。

问题 3：线程 0 和线程 1 收到的起点都是 (0, 0)，这有什么用？

它表示"从哪里开始检查"：`check_rows` 从第 `data->row` 行（第 0 行）查到最后一行，`check_columns` 从第 `data->column` 列（第 0 列）查到最后一列。这样所有线程都用同一种方式接收任务位置，符合实验文档"父线程把要检查的位置传给每个工作线程"的要求。

问题 4：每次运行，线程的执行顺序都一样吗？

不一定。线程什么时候运行由操作系统决定，每次都可能不同。但输出的顺序永远一样，因为打印发生在所有线程结束之后，由主线程按编号 0～10 依次打印。结论也永远一样，因为每个线程的检查结果只取决于数独本身，与执行顺序无关。

问题 5：用了多线程，程序就更快了吗？

对这个程序来说并不会。创建一个线程的开销远大于检查 9 个数字的开销，所以用 11 个线程，往往比单线程直接检查还要慢。本实验的目的是练习创建线程、传递参数和收集结果。只有当每个子任务的计算量足够大、机器又有多个 CPU 核心时，多线程才能明显提速。

问题 6：输出里的 "Thread 3" 是操作系统分配的线程号吗？

不是。它是程序自己的编号，也就是 `result` 和 `workers` 数组的下标。操作系统分配的线程编号保存在 `workers[i]`（`pthread_t` 类型）里，程序没有打印它。

问题 7：编译时没有加 `-pthread` 会怎样？

本项目的命令就没有加。在较老的 Linux 系统上，这样链接会报类似 `undefined reference to pthread_create` 的错误；较新的系统（glibc 2.34 及以后）已经把 pthread 合并进 C 库本身，可以直接链接，加上 `--static` 也能生成静态可执行文件。`-pthread` 仍是可移植性最好的写法，换到旧系统时记得加上。

7. 小结

要点	代码中的实现
线程分工（实验文档建议的 11 线程方案）	线程 0 执行 `check_rows`，线程 1 执行 `check_columns`，线程 2～10 执行 `check_subgrid`
创建与等待线程	`pthread_create` 创建，`pthread_join` 等待全部结束
给线程传参（实验文档 *Passing Parameters to Each Thread*）	`malloc` 一个 `parameters` 结构体写上起点，以 `void *` 交给线程，线程用完后 `free`
向主线程返回结果（实验文档 *Returning Results to the Parent Thread*）	全局数组 `result`，线程 `i` 只写 `result[i]`，因此不需要锁
检查一个区域	`region_is_valid` 用 `seen` 数组判断区域里是否恰好有 1～9